

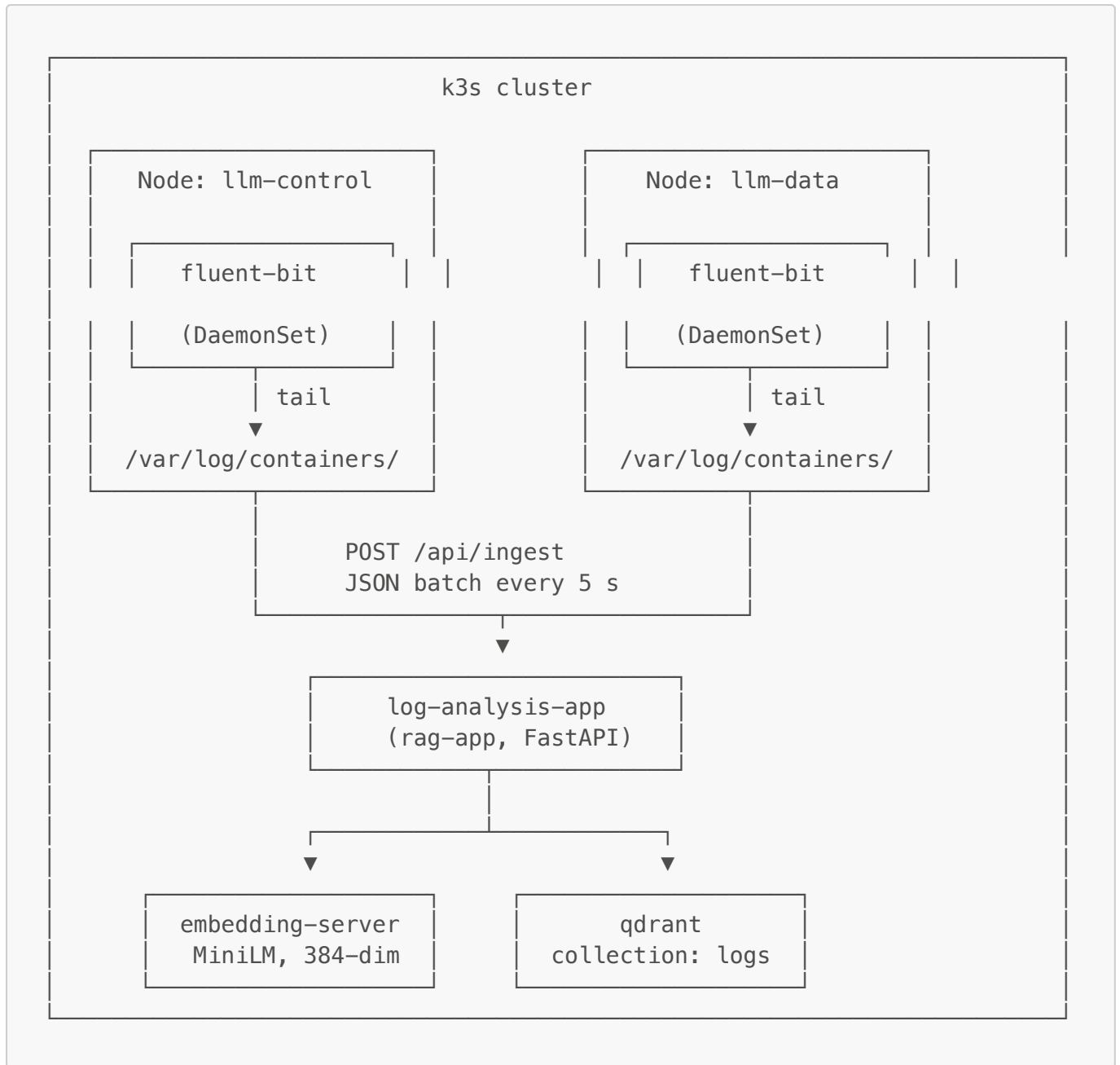
Live Log Ingestion — Fluent Bit → rag-app → Qdrant

This document describes the live log ingestion pipeline that streams cluster pod logs into the RAG vector store in near real time. With this in place, the LLM can answer questions about logs that arrived seconds ago, not just the static `sample-logs` ConfigMap that ships with the PoC.

TL;DR

A Fluent Bit DaemonSet on every node tails container logs, enriches them with Kubernetes metadata, and POSTs JSON batches every 5 seconds to a new `/api/ingest` endpoint on the rag-app. The endpoint embeds each record via the embedding-server and upserts the vectors into the existing Qdrant `logs` collection. The chat UI immediately sees the new chunks because retrieval already reads the same collection.

Architecture



Components

Component	Where	Role
Fluent Bit DaemonSet	manifests/08-fluent-bit.yaml	Tails <code>/var/log/containers/*.log</code> , parses CRI format, enriches with k8s metadata, ships JSON batches over HTTP
<code>/api/ingest</code> endpoint	images/rag-app/main.py	Receives batches, normalizes records, embeds, upserts to Qdrant
embedding-server	manifests/03-embedding-server.yaml	Returns 384-dim vectors (sentence-transformers MiniLM)
Qdrant	manifests/01-qdrant.yaml	Stores vectors + payload in the <code>logs</code> collection (cosine distance)

Per-record flow

A single log line travels through eight stages from disk to a queryable vector.

Stages 1-4: collection and enrichment

1. Container writes to stdout
/var/log/containers/grafana-6874f77485-nkf96_monitoring_grafana-...log
2026-05-04T20:52:27.328919Z stdout F level=info msg="Request OK"



2. Fluent Bit [INPUT tail]
Parser=cri splits the CRI line into structured fields:

{ time, stream, logtag, log }



3. Fluent Bit [FILTER kubernetes]
Calls the k8s API and enriches the record with:
kubernetes: { namespace_name, pod_name, container_name, ... }



4. Fluent Bit [FILTER grep]
Drops records where the `log` field is empty

Stages 5-8: ingest and vector upsert

5. Fluent Bit [OUTPUT http]
Every 5 s: POST a JSON array to
http://log-analysis-app.ai-platform.svc.cluster.local:8000
/api/ingest



6. rag-app parse_shipper_record()
Normalizes each record into the rag-app payload shape:
{ text, source = "ns/pod/container", level, timestamp }
Truncates text > 2000 chars; classifies level by regex.



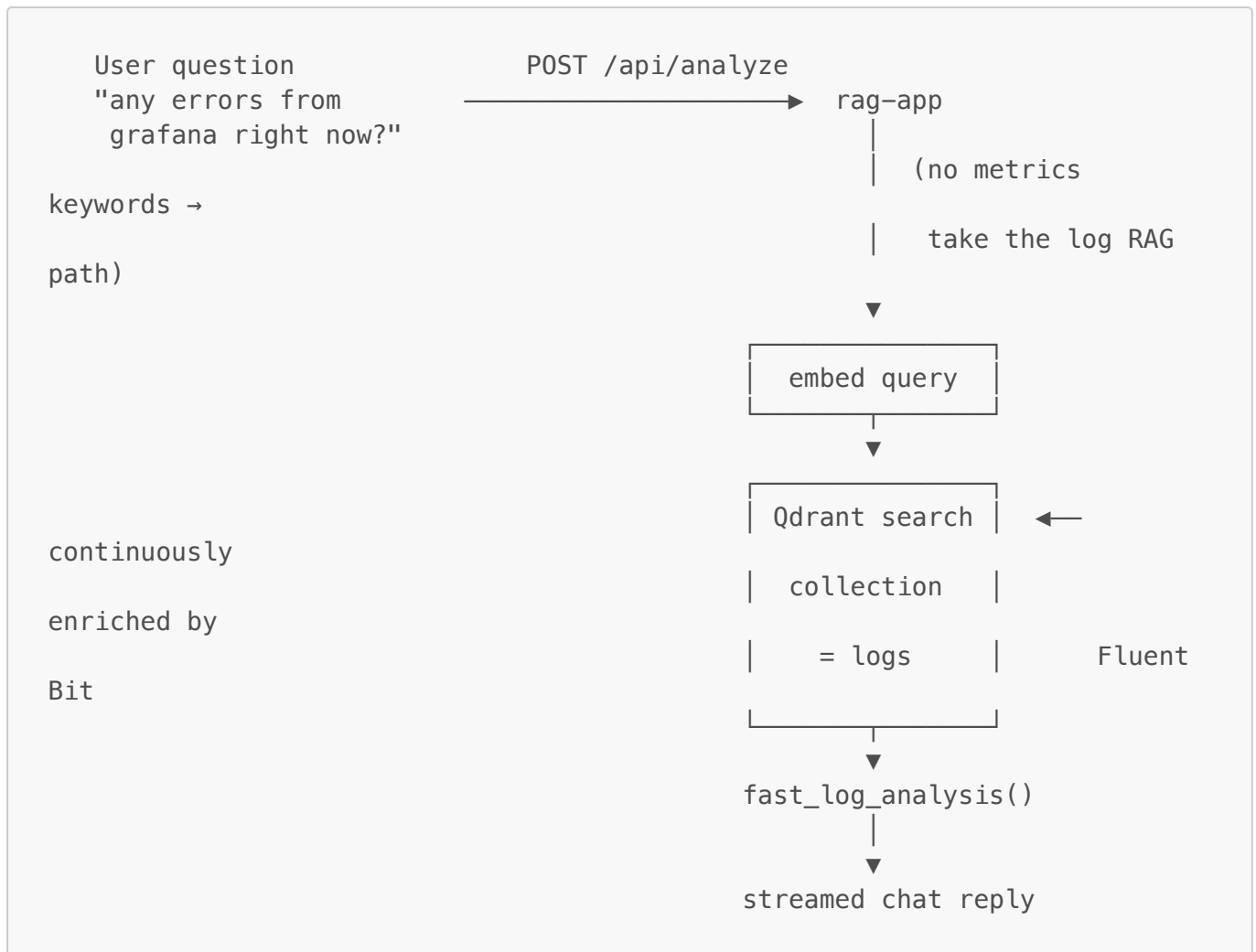
7. embedding-server POST /embed
Whole batch in one call → list of 384-dim vectors



8. Qdrant PUT /collections/logs/points
Upserts points: { id = uuid4, vector, payload }

How the chat sees live data

The retrieval path was already reading from the same Qdrant collection used by the original ingestion job, so no changes to the query side were needed.



Feedback-loop prevention

The rag-app, embedding-server, and Qdrant all produce logs themselves. If Fluent Bit shipped those logs through `/api/ingest`, every ingest call would generate more logs to ingest, and the cluster would meltdown.

Two layers of defense:

1. **Path filter** — Fluent Bit's `Tail` input only watches files matching `/var/log/containers/*_ai-platform*.log` and `*_monitoring*.log`.
2. **Exclude list** — `Exclude_Path` skips any file whose name contains `log-analysis-app`, `embedding-server`, `qdrant`, `fluent-bit`, or `log-ingestion`.

ai-platform/		monitoring/	
└─ qwen3-server	✓ shipped	└─ prometheus	✓ shipped
└─ log-ingestion	x excluded	└─ grafana	✓ shipped
└─ log-analysis-app	x excluded	└─ kube-state-metrics	✓ shipped
└─ embedding-server	x excluded	└─ node-exporter	✓ shipped
└─ qdrant	x excluded		
└─ fluent-bit	x excluded		

Everything else (kube-system, cattle-system, cert-manager, ...) is outside the two included path globs and is silently ignored.

Operations

Verify the pipeline is healthy

```
# Both DaemonSet pods Running
vagrant ssh control --command "sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml \
  kubectl get pods -n ai-platform -l app.kubernetes.io/name=fluent-bit"

# Each batch should log HTTP status=200 and {"ingested":N,"skipped":0}
vagrant ssh control --command "sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml \
  kubectl logs -n ai-platform -l app.kubernetes.io/name=fluent-bit --
  tail=20"

# Qdrant point count should grow over time
vagrant ssh control --command "sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml \
  kubectl exec -n ai-platform deploy/log-analysis-app -- \
  python3 -c 'import urllib.request,json; \

print(json.loads(urllib.request.urlopen(\"http://qdrant:6333/collections/lo
gs\").read()))[\"result\"][\"points_count\"])'"
```

Expand coverage to more namespaces

Edit the **Path** line in [manifests/08-fluent-bit.yaml](#)

to include additional globs, e.g.:

```
Path  /var/log/containers/*_ai-platform_*.log,\
      /var/log/containers/*_monitoring_*.log,\
      /var/log/containers/*_kube-system_*.log
```

Then:

```
kubectl apply -f manifests/08-fluent-bit.yaml
kubectl rollout restart daemonset/fluent-bit -n ai-platform
```

Reset the collection

If you want to drop everything and start fresh:

```
vagrant ssh control --command "sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml \
  kubectl exec -n ai-platform deploy/log-analysis-app -- \
  python3 -c 'import urllib.request; \
    r = urllib.request.Request(\"http://qdrant:6333/collections/logs\",
method=\"DELETE\"); \
    urllib.request.urlopen(r)'"
```

```
# Restart the rag-app so it recreates the collection on the next ingest
kubectl rollout restart deployment/log-analysis-app -n ai-platform
```

Troubleshooting

Symptom	Likely cause	Fix
HTTP status=503 in fluent-bit logs	embedding-server not ready	<code>kubectl get pods -n ai-platform</code> and wait for it to be Running
<code>points_count</code> not growing	Path filter doesn't match any active pods	Check <code>kubectl get pods --all-namespaces</code> against the <code>Path</code> globs
Source field is <code>live-stream</code>	<code>kubernetes</code> filter didn't enrich (RBAC issue)	Verify ServiceAccount/ClusterRoleBinding from manifests/08-fluent-bit.yaml is applied
Qdrant fills up disk	No retention set; live ingest is unbounded	Add a periodic Job that DELETES old points by timestamp, or lower the <code>Path</code> scope